CSE 711 - Compilers

Lecture 1 (Part 1)

Course Instructor: Ms. Samia Muntaha
Department of Computer Science and Engineering
University of Chittagong

Researchgate Profile

Objectives

- Role of Lexical Analyzer
- Token, Pattern and Lexeme
- Types of Errors
- Input Buffers
- Recognition of tokens
- Transition Diagrams

Role of the Lexical Analyzer

- Primary Roles
  - Scan a source program (a string) and break it up into small, meaningful units, called tokens.
  - Example: position := initial + rate * 60;
  - Transform into meaningful units: identifiers, constants, operators, and punctuation.
- Other roles:
  - Removal of comments
  - Case conversion
  - Removal of white spaces

Role of the Lexical Analyzer

x = a+b*c

Lexical Analysis

Lexemes	Tokens
x	Identifier
=	Operator
a	Identifier
+	Operator
b	Identifier
*	Operator
c	Identifier

Recognizes token using Regexes.
Eg. Regex for Identifier:
l(l+d)*|_(l+d)*
l : letter
d : digit
_ : underscore

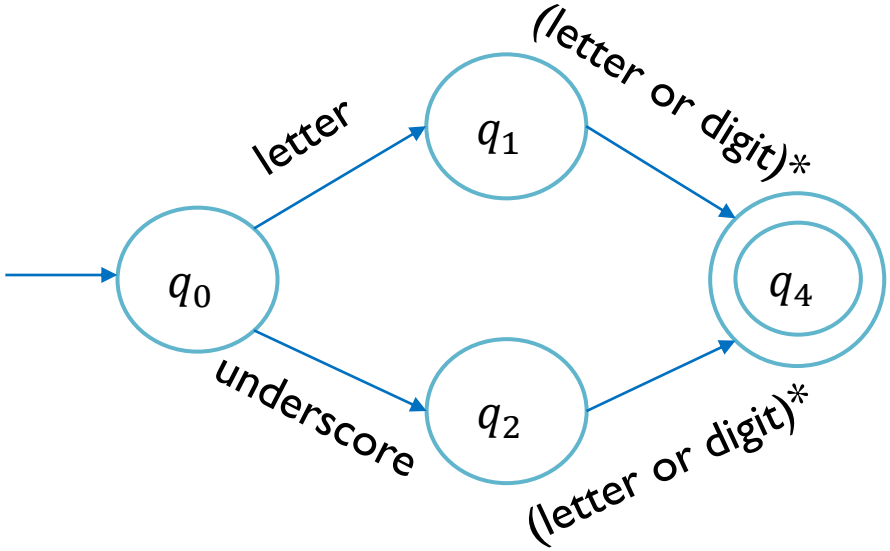
Role of the Lexical Analyzer

x = a+b*c

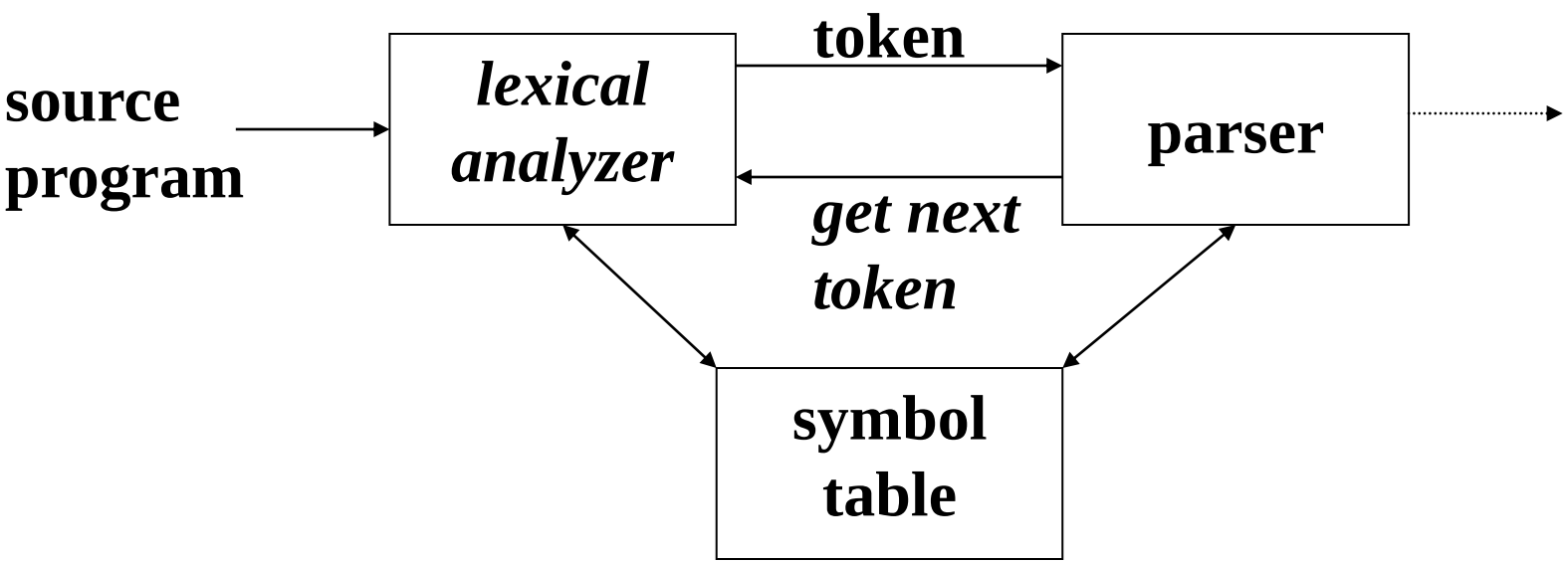
Lexical Analysis

Recognizes token using Regexes.
Eg. Regex for Identifier:
l(l+d)*|_(l+d)*
l : letter
d : digit
_ : underscore

Lexemes	Tokens
x	Identifier
=	Operator
a	Identifier
+	Operator
b	Identifier
*	Operator
c	Identifier



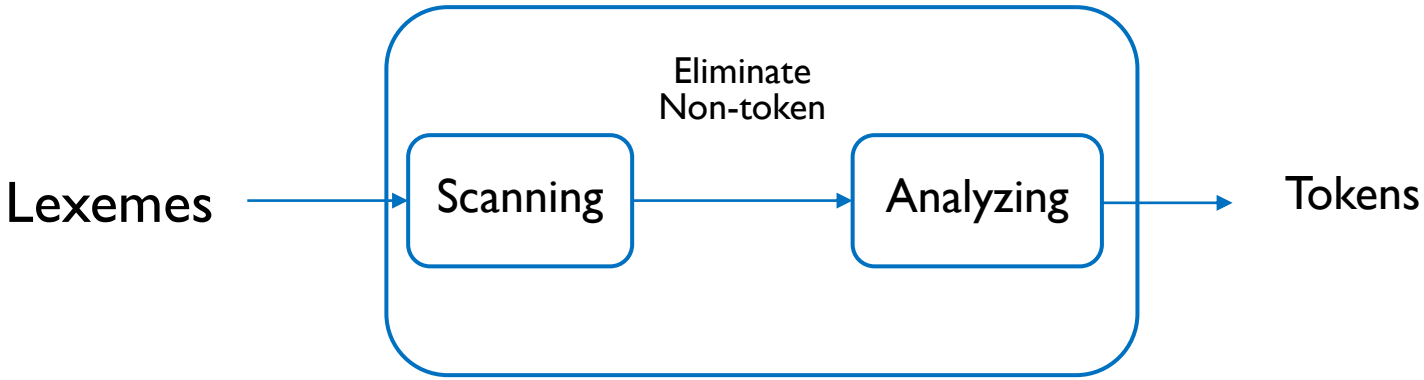
Interaction between the Lexical Analyzer and Parser



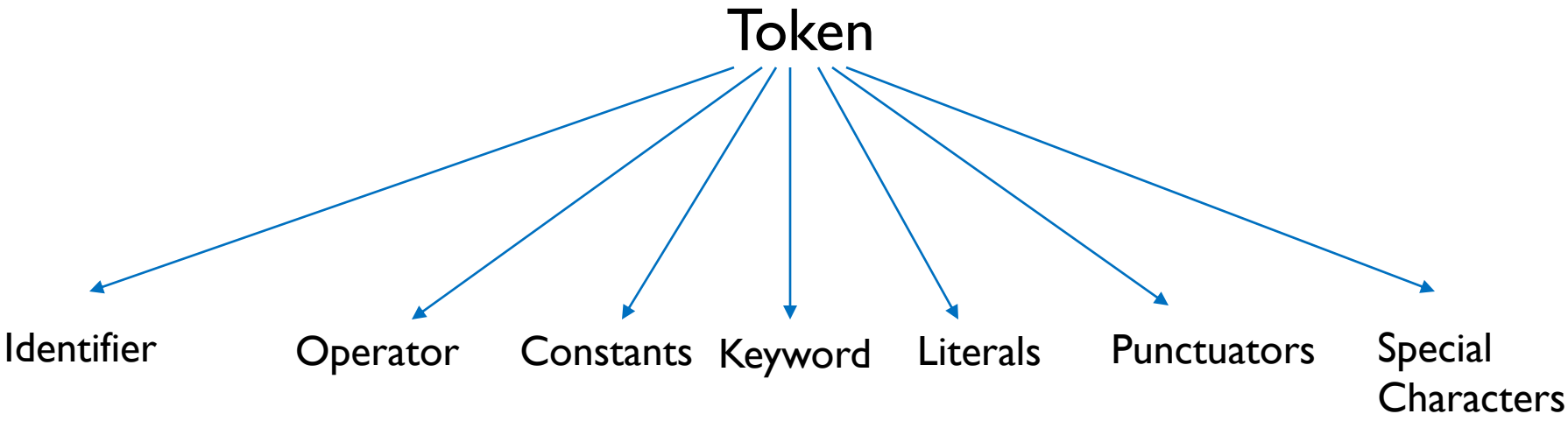
Separation of LA and Parser

- Why separate LA from parser?
  - Simpler design of both LA and parser
  - More efficient compiler
  - More portable compiler

Lexical Analyzer Functions



Token, Pattern and Lexemes



Token, Pattern and Lexemes

Token	Sample Lexemes	Informal Description of Pattern
const	const	const
if	if	characters i, f
relation	<, <=, =, < >, >, >=	< or <= or = or < > or >= or >
id	pi, count, D2	letter followed by letters and digits
num	3.1416, 0, 6.02E23	any numeric constant
literal	"core dumped"	any characters between " and " except "

Classifies Pattern

Actual values are critical. Info is :

1. Stored in symbol table
2. Returned to parser

Example

What are the tokens in the following program?

```
int main()
 {
   float a = 123.4;
   return 0;
  }
```

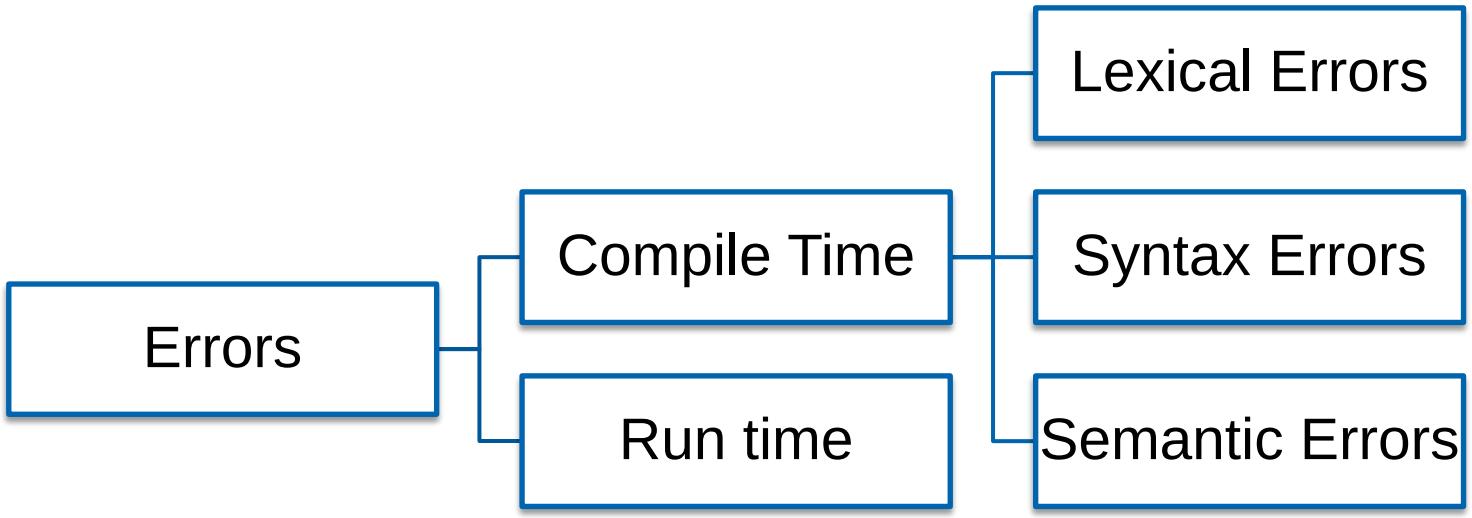
A Quick Exercise

- Count the number of tokens in the given code:

```
printf("i=%d, &i=%x",i,&i);
```

10

Types of Errors



Lexical Errors

- Too long identifiers
- Exceeding lengths of numeric constants

```
int i = 4567891;
```

- Ill-formed numeric constants

```
int i = 4567$91;
```

- Illegal characters that are absent from the source code

```
char x[ ] = “compiler”;$
```

Dealing with errors

- Panic mode recovery: delete characters from input until a matching pattern is found.
- Insert missing character.
- Replace a character with another.
- Transpose two adjacent characters

Buffer Pairs

- Lexical analyzer needs to look ahead several characters beyond the lexeme for a pattern before a match can be announced.
- In C, single-character operators like =, or < could also be the beginning of a two-character operator like ==, or <=.
- Use a function **ungetc** to push lookahead characters back into the input stream.
- Large amount of time can be consumed moving characters.

Special Buffering Technique

Use a buffer divided into two N-character halves

N = Number of characters on one disk block

One system command read N characters

Fewer than N character => eof

Code to advance forward pointer

```
if  forward at the end of first half then  begin
        reload second half ;
        forward  : =  forward  + 1;
end
else if  forward  at end of second half  then begin
          reload first half ;
          move  forward  to beginning of first half
end
else  forward  : =  forward  + 1;
```

Pitfalls:

1.This buffering scheme works quite well most of the time but with it amount of lookahead is limited.

2.Limited lookahead makes it impossible to recognize tokens in situations where the distance, forward pointer must travel is more than the length of buffer.

Problem?

- **Except at the end of buffer halves, for each advance of *forward* requires 2 tests**

```
if  forward at the end of first half then  begin
      reload second half ;
      forward  : =  forward  + 1;
end
else if  forward  at end of second half  then begin
        reload first half ;
        move  forward  to beginning of first half
end
else  forward  : =  forward  + 1;
```

Algorithm: Buffered I/O with Sentinels

			E		=		M	*	eof	C	*	*	2	eof					eof

lexeme beginning

forward (scans ahead to find pattern match)

```
forward := forward + 1 ;
if forward ↑ = eof then begin
    if forward at end of first half then begin
        reload second half ;   ← Block I/O
        forward := forward + 1
    end
    else if forward at end of second half then begin
        reload first half ;   ← Block I/O
        move forward to beginning of first half
    end
    else / * eof within buffer signifying end of input * /
        terminate lexical analysis
end        2nd eof ⇒ no more input !
```

We extend each buffer half to hold a sentinal [*eof*] character at the end

Token Recognition

How can we use concepts developed so far to assist in recognizing tokens of a source language ?

<u>Assume Following Tokens:</u>

if, then, else, relop, id, num

We can express patterns using regular expressions.

<u>Given Tokens, What are Patterns ?</u>

if → **if**

then → **then**

else → **else**

relop → < | <= | > | >= | = | <>

digit → [**0-9**]

letter → [**A-Za-z**]

id → **letter** (**letter** | **digit**)*

num → **digit** $^{+}$ (. **digit** $^{+}$) ? (E(+ | -) ? **digit** $^{+}$) ?

Grammar:

stmt → |**if** ***expr*** **then** ***stmt***
|**if** ***expr*** **then** ***stmt*** **else** ***stmt***
|∈

expr → ***term*** **relop** ***term*** | ***term***

term → **id** | **num**

What Else Does Lexical Analyzer Do?

Scan away *blanks*, new lines, tabs

Can we Define Tokens For These?

blank $\rightarrow$ **blank**

tab $\rightarrow$ **tab**

newline $\rightarrow$ **newline**

delim $\rightarrow$ **blank | tab | newline**

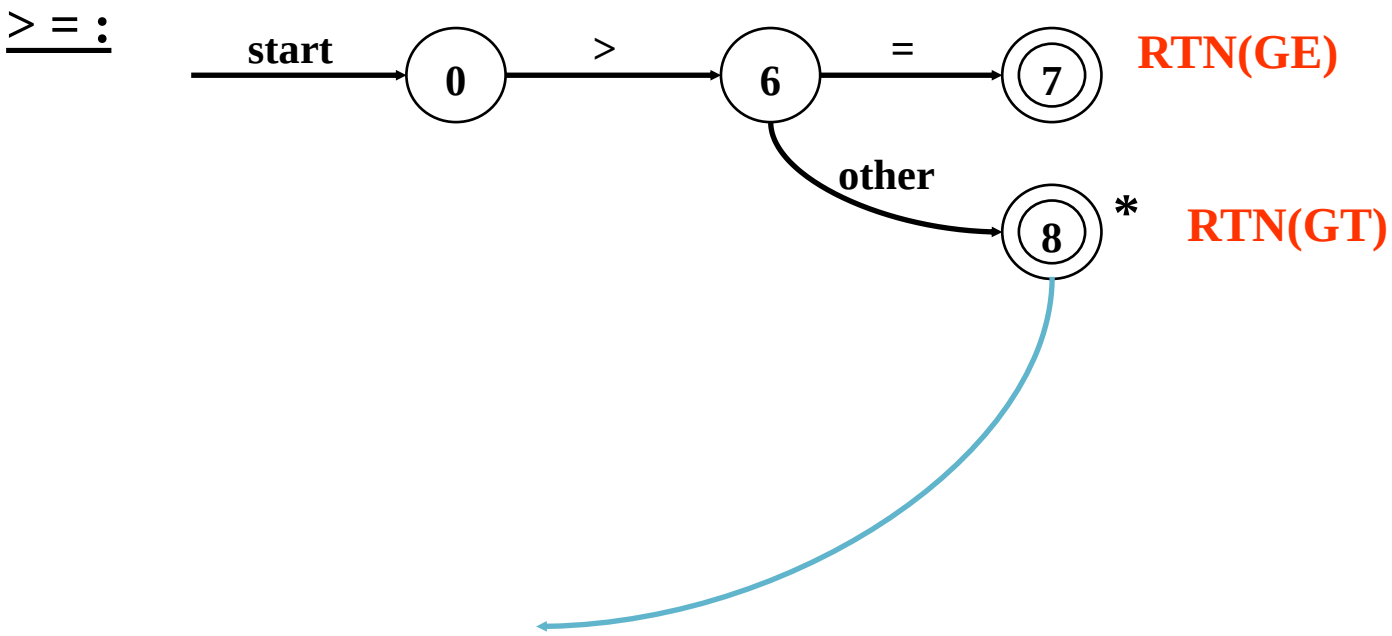
ws $\rightarrow$ **delim** $^{+}$

In these cases no token is returned to parser

Constructing Transition Diagrams for Tokens

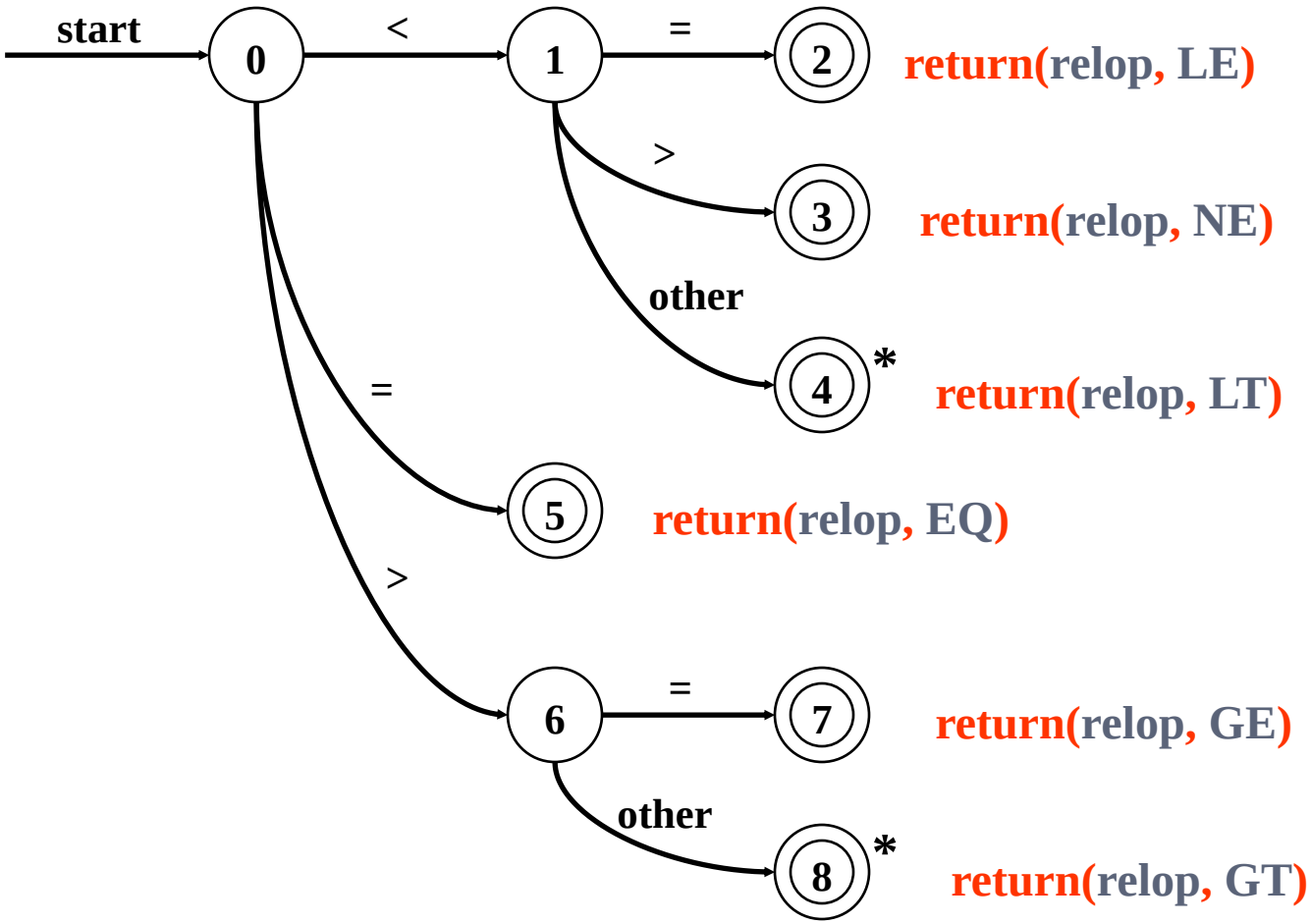
- **Transition Diagrams (TD) are used to represent the tokens**
- **As characters are read, the relevant TDs are used to attempt to match lexeme to a pattern**
- **Each TD has:**
  - **States : Represented by Circles**
  - **Actions : Represented by Arrows between states**
  - **Start State : Beginning of a pattern (Arrowhead)**
  - **Final State(s) : End of pattern (Concentric Circles)**
- **Each TD is Deterministic (assume) - No need to choose between 2 different actions !**

Example TDs



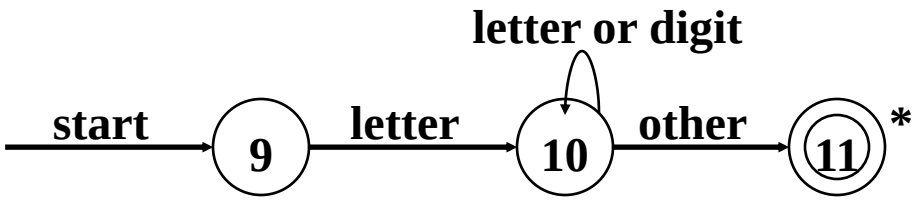
We've accepted ">" and have read one extra char that must be unread.

Example : All RELOPs



Example TDs : id and delim

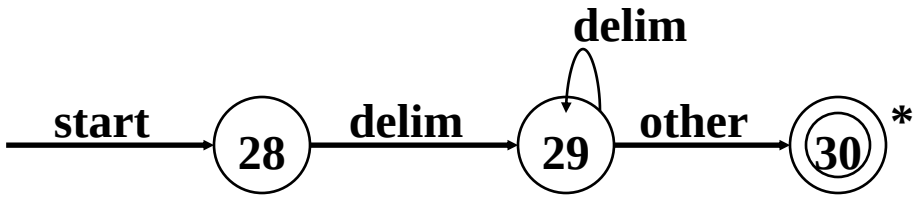
id :



return(get_token(), install_id())

Either returns ptr or "0" if reserved

delim :



Question: **Why are there no TDs for `then, else, if` ?**

What Else Does Lexical Analyzer Do?

All Keywords / Reserved words are matched as ids !!

- **So recognizing keywords and identifiers presents a problem.**
- **There are two ways that we can handle reserved words that look like identifiers.**

What Else Does Lexical Analyzer Do?

First Approach (Preferred):

- **Install the reserved words in the symbol table initially.**
- **After the match, the symbol table or a special keyword table is consulted.**
- **Keyword table contains string versions of all keywords and associated token values**

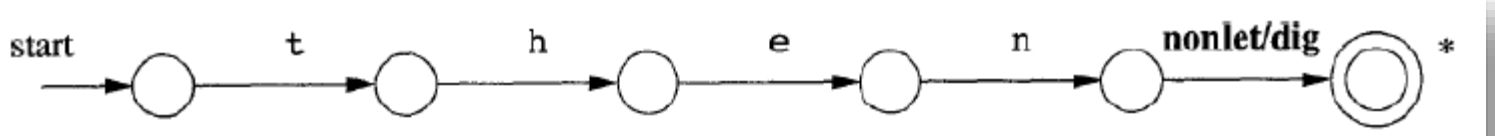
if	**15**
then	**16**
begin	**17**
...	**...**

- **When a match is found, the token is returned, along with its symbolic value, i.e., "then", 16**
- **If a match is not found, then it is assumed that an id has been discovered**

What Else Does Lexical Analyzer Do?

Second Approach :

- **Create separate transition diagrams for each keyword.**



Hypothetical transition diagram for the keyword then

Example TDs : Unsigned Number

digit → 0 | 1 | 2 | … | 9

digits → digit digit*

1240, 39.45, 6.33E15, or 1.578E-41

optional_fraction → . digits | ∈

optional_exponent → (E (+ | -| ∈) digits) | ∈

num → digits optional_fraction optional_exponent

Shorthand

digit → 0 | 1 | 2 | … | 9

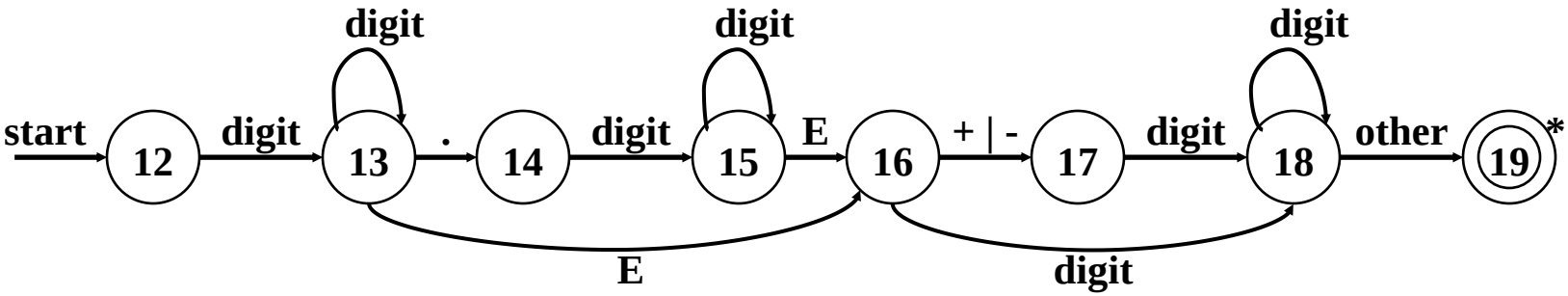
digits → digit$^+$

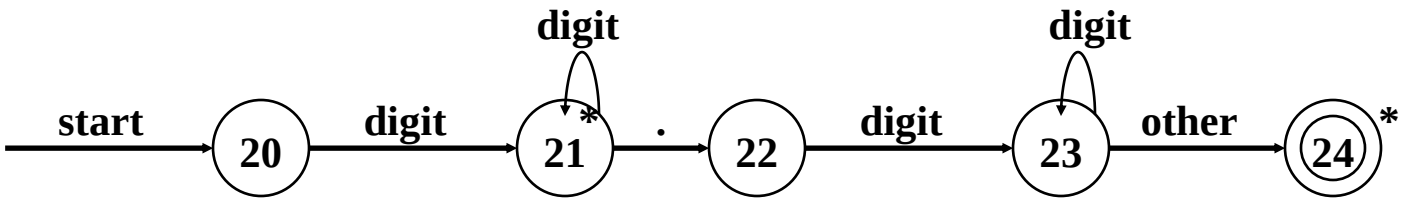
optional_fraction → (. digits) ?

optional_exponent → (E (+ | -) ? digits) ?

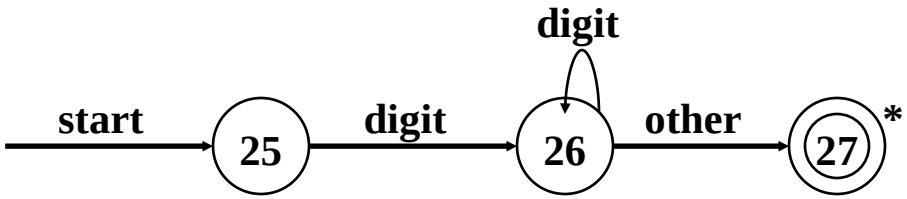
num → digits optional_fraction optional_exponent

Example TDs : Unsigned #s





return(num, install_num())



Questions: **Is ordering important for unsigned #s ?**

Implementing Transition Diagrams

A sequence of transition diagrams can be converted into a program to look for the tokens specified by the grammar

Each state gets a segment of code

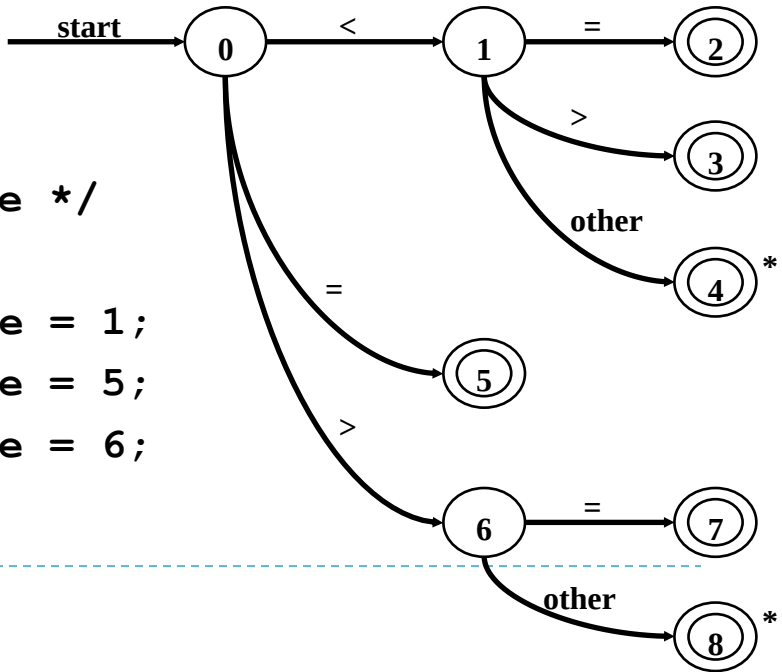
```
FUNCTIONS USED
    •nextchar(),
    •retract(),
    •install_num(),
    •install_id(),
    •gettoken(),
    •isdigit(),
    •isletter(),
    •recover()
```

Implementing Transition Diagrams

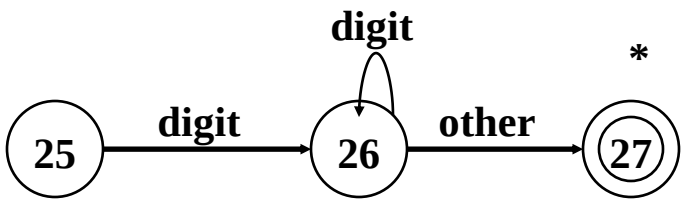
```
int state = 0, start = 0

lexeme_beginning = forward;
token nexttoken()
{   while(1) {
        switch (state) {
        case 0:   c = nextchar();
            /* c is lookahead character */
            if (c== blank || c==tab || c== newline) {
                state = 0;
                lexeme_beginning++;
                    /* advance
                  beginning of lexeme */
            }
            else if (c == '<') state = 1;
            else if (c == '=') state = 1;
            else if (c == '>') state = 6;
            else state = fail();
            break;
            … /* cases 1-8 here */
```

repeat until a "return" occurs



Implementing Transition Diagrams, II




```
.............
case 25;  c = nextchar();
          if (isdigit(c)) state = 26;
          else state = fail();
          break;
case 26;  c = nextchar();
          if (isdigit(c)) state = 26;
          else state = 27;
          break;
case 27;  retract(1); lexical_value = install_num();
          return ( NUM );
.............
```

advances `forward`

Case numbers correspond to transition diagram states !

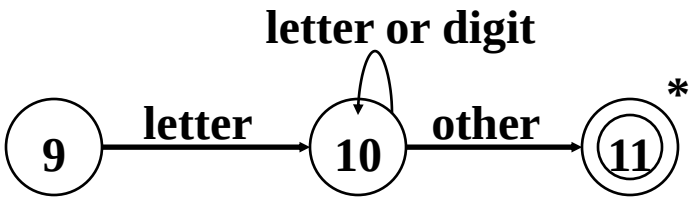
retracts `forward`

looks at the region `lexeme_beginning ... forward`

Implementing Transition Diagrams, III

```
.............
 case 9:   c = nextchar();
           if (isletter(c)) state = 10;
           else state = fail();
           break;
 case 10;  c = nextchar();
           if (isletter(c)) state = 10;
           else if (isdigit(c)) state = 10;
           else state = 11;
           break;
 case 11;  retract(1); lexical_value = install_id();
           return ( gettoken(lexical_value) );
.............
```

reads token name from ST



When Failures Occur:

```
int fail()
{
      forward = lexeme beginning;
      switch (start) {
         case 0:   start = 9;  break;
         case 9:   start = 12; break;
         case 12:  start = 20; break;
         case 20:  start = 25; break;
         case 25:  recover();  break;
         default:   /* lex error */
      }
      return start;
}
```

Switch to next transition diagram

References

- https://www.nesoacademy.org/cs/12-compiler-design
- Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Addison-Wesley, 2006